ML/ AI Research Internship Take Home Task

1. **Framework: TRL** – Hugging face Transformer reinforcement learning. TRL has a seamless integration with QLoRA and 4 Bit (BitAndBytes), GRPO native support, has Simple python callables as well.

2. **Benchmark: tau2 bench**, and Synthetic sample size=200

3. **Environment Architecture Creation:**
   To ensure maintainable and scalable training environment this work contains **six major components.**

   1. model_setup.py
   Loads Qwen2.5-7B-Instruct with 4-bit NF4 quantization (BitsAndBytes) and applies LoRA adapters (r=16, alpha=32) to attention and MLP projection layers.

   2. dataset_prep.py:
   Ingests the tau2-bench HuggingFace dataset, runs synthetic task generation for edge-cases, and formats all examples into chat-template prompts.

   3. reward_func.py
   The core alignment module. It defines the mathematical reward signals, featuring both format-based heuristics (verifying JSON structure) and correctness heuristics (verifying the logical trajectory of tool choices).

   4. grpo_config.py
   Centralizes all RLVR hyperparameters: learning rate, beta (KL penalty), generation count, completion length limits, and LoRA settings.

   5. trainer.py
   Orchestrates the active training loop. Manages gradient accumulation, memory-efficient loading, and checkpoint saving.

   6. evaluation.py
   An isolated inference script that loads the fine-tuned model adapters over the base model to validate trajectory accuracy against unseen test prompts which produces the improved difference before vs after training.

3a. State / Observation Space

The observation presented to the RL agent at each step is a tokenized chat-formatted prompt constructed from three components:

1. System Prompt: Contains Tool registry: names, signatures, and descriptions of all available API functions.
2. User Utterance: contains the customer service request referencing an order ID and describing an issue.
3. Generation Prompt: The assistant turn start token, signaling the model to begin generating. The Special token: (<|im_start|>assistant).

3b. Action Space

The action space is the set of all valid next-token predictions over the model's vocabulary. However, the environment constrains the meaningful action space to well-formed JSON tool-call sequences. An action is considered valid if and only if it satisfies both conditions below:

I. **Structural validity:** Output is parse-able JSON containing a 'name' key and optionally an 'arguments' key.
II. **Semantic validity:** The tool name exists in the available tool registry and the argument types match the tool signature

3c. Reward Function Design

Two reward functions are composed additively. This design allows each signal to target a distinct failure mode independently.

I. **Reward Function 1 — Structural Format Reward (format_reward_func)**
Targets: conversational drift, malformed output, missing JSON structure.

II. **Reward Function 2 — Correctness / Trajectory Reward (correctness_reward_func)**
Targets: wrong tool selection, hallucinated tool calls, incorrect sequencing.

4. **Model Selection and training plan:**

Base Model Selection	**Qwen 2.5 7B Instruct** *Soorya's reasoning:* *It's a robust zero-shot reasoning and native understanding of tool - calling schemas.*
RL Algorithm	**GRPO – Group Relative Policy Optimization** *Soorya's Reasoning:* *Eliminates the need for a separate critic model by normalizing rewards across multiple sampled generations.*
Dataset Configuration	- Sample: 200 - Train / Val / Test: 160 / 20 / 20 - Curriculum: N/A (Uniform Sampling)
Hyperparameters	- Learning Rate: 1e - 5 - Batch Size: 1 (with gradient accumulation steps) - Epochs: 75 - Optimizer: Adamw_torch - LR Scheduler: Cosine Decay - LoRA Rank: 16 - LoRA Alpha: 32 - GRPO: 4 per prompt
Training Reward Design	- Reward: **+0.5 - +1.0** for valid JSON arrays and correct tool selection. - Penalty: **-0.5 to -2.0** for malformed formatting, missing parameters, or conversational deviation.
Target Metric	1. Mean Reward Progression 2. KL Divergence from Reference Model 3. Tool Execution Accuracy

Outcome:

GitHub Link: https://github.com/Soorya2201/AfterQuery_RLVR_Submission
Collaborator Access: michael@afterquery.com

The model successfully learned to bypass conversational filler, translating user intents directly into valid, parsable JSON tool sequences as enforced by the format_reward_func

```
--- Final Model Output ---
{
  "action": "get_order_status",
  "order_id": "X999"
}
{
  "action": "refund_order",
  "order_id": "X999"
}
```

75 Epoch Training losses:

Step	Training Loss
2	0.000000
4	0.019172
6	-0.071801
8	0.076295
10	0.077294
12	0.000007
14	0.000008
16	0.048401
18	0.000007
20	0.020570
22	0.024917
24	0.000018
26	-0.005460
28	0.000028
30	0.054892
32	0.089980
34	0.038442
36	0.054549
38	0.000042
40	0.040780
42	0.000008
44	0.000016
46	0.127417
48	-0.003861
50	0.000020
52	-0.001266
50	0.000020
52	-0.001266
54	0.000026
56	0.020515
58	0.000012
60	0.000013
62	0.000010
64	0.024267
66	0.050380
68	0.037780
70	0.000017
72	0.023710
74	0.000026

Optional Strech Goals Completed

I. **Synthetic task generation: Yes**

While running main.py, The terminal's argparse feature will ask for what dataset to be used from 3 options:
1. tau2-bench (only)
2. Synthetic data based on tau2 Bench (only)
3. tau2-bech + Synthetic data (both)

II. **Scaling: Yes**

Scaleup: We can now introduce higher sparse data's, decrease the learning rate to 5e -6 or 1e – 6 because large models suffer from catastrophic forgetting. Reduce GRPO generations to 4 as it's

now computationally expensive with large models. Increase LoRA expressiveness to 32 or 64 to capture broader representation of larger weight matrices.

Scale down: simplified schemas and increased data size (perform synthetic data generation), one tool at a time. Increased learning rate to 3e-5 or 5e-5. Increase GRPO generations (to 8 or 16) and batch sizes (to 4 or 8), as now we have plenty of VRAM.

Optional Strech Soorya's Addition

Rigorous A/B Evaluation: Yes

Rather than reporting training loss alone, I have created a dedicated evaluation.py script that performs a side-by-side comparison of the base model and the RLVR fine-tuned model on identical held-out prompts.
This surfaces the behavioral delta that training loss obscures specifically whether the model has shifted from conversational responses to structured tool-call outputs

> **Why:** When I was working as ML engineer, I understood that Loss alone is not a reliable indicator of behavioral improvement for any model. The A/B evaluation is one of the meaningful measures of whether the reward signal shaped the desired behavior.

Explicit GPU Memory Management: Yes

To prevent CUDA out-of-memory errors between the training and evaluation phases, explicit garbage collection is implemented using Python's gc.collect() and torch.cuda.empty_cache()

Soorya's Work Metrics:
Time taken to complete: 5 Hrs.
Difficulty level: Moderately Easy
Interest to work with AfterQuery: Beyond Infinity!

Thank you michael@afterquery.com for your time!